

Viajaí Bolso

Documentação Técnica e Funcional do Projeto

App de controle de gastos em viagens — Flutter

Projeto desenvolvido como trabalho de conclusão do 3º período do curso de Análise e Desenvolvimento de Sistemas, em fase de evolução para se tornar um aplicativo funcional completo.

Documento gerado e atualizado automaticamente a cada alteração relevante no código.

1. Visão Geral do Projeto

O **Viajaí Bolso** é um aplicativo mobile (Flutter) para controle financeiro de viagens: o usuário cadastra viagens — cada uma com destino, período e orçamento — e registra despesas associadas a cada viagem, categorizadas (alimentação, transporte, etc.), acompanhando o quanto já foi gasto em relação ao orçamento definido.

O projeto nasceu como protótipo de conclusão do 3º período do curso de Análise e Desenvolvimento de Sistemas e agora está em evolução para se tornar um aplicativo funcional completo, a ser apresentado como projeto final do curso. Este documento acompanha essa evolução: a cada mudança relevante no código, uma nova seção é adicionada ao **Histórico de Alterações** (seção 7), mantendo sempre a descrição das seções anteriores atualizada com o estado mais recente do app.

1.1 Principais dependências (pubspec.yaml)

Pacote	Finalidade
supabase_flutter	Cliente oficial do Supabase: autenticação, banco de dados (Postgres) e armazenamento de arquivos
image_picker / image_picker_for_web	Seleção de fotos (foto de perfil e foto da viagem), enviadas ao Supabase Storage
intl	Suporte a internacionalização/formatação (incluído, uso pontual)

1.2 Arquitetura em camadas

O código em `lib/` está organizado em três camadas, sem uso de gerenciador de estado externo (Provider, Riverpod, Bloc) — o controle de estado é feito manualmente com `setState`:

- **constants.dart** — configurações globais: URL/chave do projeto Supabase e paleta de cores do app.
- **models/** — classes de dados puras (Despesa, Categoria, Usuario, Viagem), cada uma com método `fromJson` para converter a resposta do banco.
- **services/** — camada de acesso ao Supabase (AuthService, ViagemService, DespesaService, DashboardService, StorageService), usando o cliente `supabase_flutter`.
- **screens/** — telas do app, que chamam os services diretamente (não há camada intermediária de controller/repository).

Atualização (v2): o app não depende mais de um servidor próprio — desde a migração para o Supabase (seção 7, versão 2), o backend é hospedado, acessado sempre via HTTPS, com a segurança de cada usuário só acessar os próprios dados garantida por políticas de Row Level Security (RLS) diretamente no banco (ver `supabase/schema.sql`).

1.3 Acesso direto ao app (versão web)

Atualização (v3): a cada atualização enviada ao repositório, o GitHub Actions gera automaticamente a versão web do app e publica no GitHub Pages, gerando um link direto para acesso pelo navegador, sem necessidade de instalar nada — útil para demonstração na banca (ver seção 7, versão 3):

- <https://viajaibolso-design.github.io/viaja-ai-bolso/>

2. Modelos de Dados (lib/models)

2.1 Despesa e Categoria — `despesa.dart`

Representa uma despesa individual vinculada a uma viagem e a uma categoria.

Campo (Despesa)	Tipo	Descrição
idDespesa	String?	Identificador único (uuid gerado pelo Supabase)
descricao	String	Descrição da despesa
valor	double	Valor gasto
data / hora	String / String?	Data e hora do gasto
formaPagamento	String	Ex.: Cartão de crédito, Débito, Dinheiro, Pix, Outros
idViagem / idCategoria	String	Referências (uuid) à viagem e categoria associadas
categoria / icone	String? / String?	Nome e ícone da categoria (via join com categorias)

A classe **Categoria** representa uma categoria de despesa (id, nome, ícone), lida do banco.

2.2 Usuario — `usuario.dart`

Campo	Tipo	Descrição
idUsuario	String	Identificador do usuário (uuid do Supabase Auth)
nome / email	String	Dados básicos de cadastro
fotoUrl	String?	URL pública da foto de perfil no Supabase Storage
moedaPadrao	String	Moeda padrão do usuário (default: 'BRL')

2.3 Viagem — `viagem.dart`

Campo	Tipo	Descrição
idViagem	String?	Identificador da viagem (uuid)
nome / destino	String	Nome e destino da viagem
fotoUrl	String?	URL pública da foto da viagem no Supabase Storage
dataInicio / dataFim	String	Período da viagem (AAAA-MM-DD)
orcamento	double	Orçamento definido para a viagem
totalGasto / percentualGasto	double	Valores agregados, calculados pela view <code>viagens_resumo</code> no banco
idUsuario	String?	Dono da viagem

3. Serviços — Integração com o Supabase (lib/services)

Desde a versão 2 (seção 7), os serviços não fazem mais requisições HTTP a um servidor próprio: eles usam o cliente `supabase_flutter` para falar direto com a autenticação, o banco Postgres e o armazenamento de arquivos do projeto Supabase. A segurança — cada usuário só acessa os próprios dados — é garantida por políticas de Row Level Security (RLS) no banco, não por checagens manuais no app (ver `supabase/schema.sql`).

3.1 AuthService — autenticação e perfil

Método	Recurso do Supabase	Função
<code>cadastrar(nome, email, senha)</code>	<code>auth.signUp</code>	Cria uma nova conta (dispara e-mail de confirmação)
<code>login(email, senha)</code>	<code>auth.signInWithPassword</code>	Autentica o usuário
<code>logoff()</code>	<code>auth.signOut</code>	Encerra a sessão (local e no Supabase)
<code>recuperarSenha(email)</code>	<code>auth.resetPasswordForEmail</code>	Envia código de recuperação por e-mail
<code>redefinirSenha(...)</code>	<code>auth.verifyOTP + auth.updateUser</code>	Valida o código e define a nova senha
<code>getPerfil(idUsuario)</code>	tabela <code>profiles</code>	Busca os dados do perfil
<code>atualizarPerfil(...)</code>	tabela <code>profiles + auth.updateUser</code>	Atualiza nome, e-mail, moeda e/ou foto
<code>alterarSenha(...)</code>	<code>auth.signInWithPassword + auth.updateUser</code>	Reautentica e troca a senha
<code>uploadFotoPerfil(...)</code>	Storage (bucket fotos)	Envia a foto de perfil e devolve a URL pública

3.2 ViagemService, DespesaService e DashboardService

Classe / Método	Recurso do Supabase	Função
<code>ViagemService.listar(idUsuario)</code>	view <code>viagens_resumo</code>	Lista as viagens do usuário (já com totais calculados)
<code>ViagemService.cadastrar / atualizar / remover</code>	tabela <code>viagens</code>	CRUD de viagens
<code>DespesaService.listar(idViagem)</code>	tabela <code>despesas (+ join categorias)</code>	Lista despesas de uma viagem
<code>DespesaService.listarCategorias()</code>	tabela <code>categorias</code>	Lista categorias disponíveis
<code>DespesaService.cadastrar / atualizar / remover</code>	tabela <code>despesas</code>	CRUD de despesas
<code>DashboardService.getDashboard(idUsuario)</code>	consultas combinadas	Monta viagem ativa, gastos de hoje, maior gasto e últimas despesas

Classe / Método	Recurso do Supabase	Função
StorageService.upload(...)	Storage (bucket fotos)	Upload genérico de fotos (usado por viagens e perfil)

Observação técnica: a checagem de dono dos dados (usuário só vê as próprias viagens/despesas) agora é feita pelo banco via RLS, não pelo código do app — isso reduz o risco de um bug no client expor dados de outro usuário. Erros ainda são tratados de forma genérica nas telas (mensagem fixa "Erro de conexão com o servidor"); mensagens mais específicas por tipo de erro continuam sendo um ponto de melhoria.

4. Telas e Funcionalidades (lib/screens)

SplashScreen

Tela inicial. Exibe a logo por 2 segundos e verifica se há uma sessão ativa no Supabase Auth. Se houver, leva direto ao mainScreen; senão, à tela de Login.

LoginScreen

Login com e-mail e senha, opção de mostrar/ocultar senha, links para "Esqueci minha senha" e "Cadastre-se". Em caso de sucesso, o Supabase Auth já mantém a sessão salva automaticamente e o app abre o mainScreen.

CadastroScreen

Cadastro de novo usuário (nome, e-mail, senha, confirmação de senha) com aceite obrigatório dos Termos de Uso (texto exibido na própria tela).

RecuperarSenhaScreen

Fluxo de recuperação de senha em duas etapas: solicitar código por e-mail e, em seguida, informar o código recebido junto com a nova senha.

MainScreen

Shell de navegação principal, com barra inferior de 4 abas: Início (Dashboard), Viagens, Despesas e Perfil.

DashboardScreen

Tela inicial pós-login: saudação personalizada, resumo da viagem ativa (gasto vs. orçamento, com indicador circular de progresso), gastos do dia, maior gasto e lista das despesas mais recentes. Suporta atualizar puxando a tela (pull-to-refresh).

ViagensScreen

Lista todas as viagens cadastradas em cards (foto, nome, destino, datas, progresso do orçamento), com opções de editar e excluir cada viagem, e botão para cadastrar uma nova.

NovaViagemScreen

Formulário de criação/edição de viagem: foto (câmera/galeria), nome, destino, datas de início e fim, e orçamento.

DespesasScreen

Seleciona uma viagem e lista suas despesas, com filtro por categoria. Permite cadastrar, editar e excluir despesas por um formulário em painel deslizante, incluindo categoria, descrição, valor, data e forma de pagamento. Mostra o total das despesas filtradas.

PerfilScreen

Exibe e permite editar os dados do usuário (foto, nome, e-mail), alterar a senha e encerrar a sessão (logout) via Supabase Auth.

5. Fluxo de Navegação

Resumo do fluxo entre as telas do aplicativo, a partir da inicialização:

- **Splash** → verifica sessão salva → **Login** (se não houver sessão) ou **MainScreen** (se já houver uma sessão salva).
- **Login** → sucesso leva ao **MainScreen**; "Cadastre-se" leva ao **Cadastro**; "Esqueci minha senha" leva à **Recuperação de Senha**.
- **MainScreen** organiza 4 abas fixas: **Dashboard** | **Viagens** (→ Nova/Editar Viagem) | **Despesas** (formulário em painel deslizante) | **Perfil**.
- **Perfil** → **Sair da conta** encerra a sessão e retorna à tela de **Login**, limpando todo o histórico de navegação.

6. Estado Atual e Pontos de Evolução

Como protótipo acadêmico, o app já demonstra o fluxo completo de cadastro, login, gestão de viagens e despesas. Os pontos abaixo estão mapeados como próximos passos para transformá-lo em um app funcional completo — úteis tanto para o planejamento das próximas entregas quanto para a banca de apresentação:

- **Resolvido na v2:** o backend deixou de ser um servidor fixo em IP local — agora é o Supabase, acessado via HTTPS, com autenticação e regras de acesso reais (RLS).
- **Resolvido na v3:** o app passou a ter um link direto e público, gerado automaticamente a cada atualização do código (GitHub Actions + GitHub Pages) — não é mais necessário instalar nada para demonstrá-lo. Primeiro deploy já confirmado funcionando.
- **Parcialmente testado:** o app já foi executado de verdade pela primeira vez fora deste ambiente — cadastro de usuário funcionou e o e-mail de confirmação do Supabase chegou normalmente. Ainda faltam testar login, upload de foto e o CRUD completo de viagens/despesas antes da apresentação.
- Por padrão, o Supabase exige confirmação de e-mail para novas contas — vale revisar essa configuração no painel do projeto (Authentication) para decidir se isso é desejável na demonstração.
- Tratamento de erros de rede ainda é genérico (mensagem fixa), sem diferenciar tipos de erro.
- Algumas opções da interface ainda são placeholders sem função: botão "Ver todas" no Dashboard, filtro de despesas, e os itens "Moeda padrão", "Configurações", "Sobre o app" e "Suporte" na tela de Perfil.
- Termos de Uso exibidos no Cadastro são apenas um texto estático, sem uma tela própria de conteúdo legal.
- Não há persistência local (offline) dos dados de viagens/despesas — o app depende de conexão com o Supabase.
- Não há rotas nomeadas nem testes automatizados no projeto até o momento.

7. Histórico de Alterações

Esta seção registra, em ordem cronológica, as mudanças feitas no projeto a partir desta documentação — servindo como registro de evolução para a apresentação final.

Versão 1 — Estado inicial do protótipo (baseline)

Data: 27/08/2026

Documentação inicial gerada a partir do código-fonte existente do projeto (entregue ao final do 3º período), antes do início de novas implementações. Estabelece o ponto de partida do histórico de mudanças que será usado para a apresentação final do curso.

- Levantamento completo da arquitetura atual (models, services, screens)
- Nenhuma alteração de código foi feita nesta etapa — apenas documentação do estado existente

Versão 2 — Migração do backend para o Supabase

Data: 03/09/2026

O backend próprio do protótipo (servidor fixo em 192.168.0.9:5000) foi substituído pelo Supabase: banco de dados Postgres hospedado, autenticação (login, cadastro, recuperação de senha) e armazenamento de arquivos (fotos de perfil e de viagens), tudo acessado via HTTPS. O app Flutter agora fala direto com o Supabase pelo pacote oficial `supabase_flutter`, sem depender de nenhum servidor próprio rodando. Também foi configurado um repositório Git local no projeto para acompanhamento das mudanças pelo VSCode.

- Criado o schema do banco (`supabase/schema.sql`): tabelas `profiles`, `categorias`, `viagens` e `despesas`, com segurança por usuário via Row Level Security (RLS) e bucket de Storage para fotos
- `AuthService` reescrito para usar Supabase Auth (login, cadastro, logoff, recuperação de senha por código, alteração de senha e perfil)
- `ViagemService`, `DespesaService` e `DashboardService` reescritos para consultar o Postgres do Supabase diretamente, respeitando as políticas de RLS
- Fotos de perfil e de viagens passam a ser enviadas para o Supabase Storage (antes eram salvas como texto base64 dentro dos próprios registros)
- Identificadores (usuário, viagem, despesa, categoria) passam de números inteiros para UUID, seguindo o padrão do Supabase
- Removida a dependência de `SharedPreferences` para controlar a sessão — o Supabase Auth já mantém isso automaticamente
- Configurado repositório Git local no projeto (commits com histórico de mudanças, visível pelo VSCode)
- Ainda não testado em um app rodando de verdade (não foi possível compilar o Flutter neste ambiente) — recomenda-se rodar o app e testar cadastro/login/CRUD antes da apresentação

Versão 3 — Publicação da versão web (link direto de acesso)

Data: 09/09/2026

Foi criado um fluxo automático (GitHub Actions) que gera a versão web do app a cada atualização enviada ao repositório e publica o resultado no GitHub Pages. Com isso, o projeto passa a ter um link direto, acessível por qualquer navegador, sem precisar instalar nada — ideal para demonstrar o app durante a apresentação do TCC.

- Criado o workflow `.github/workflows/deploy.yml`, que instala o Flutter, roda 'flutter pub get' e 'flutter build web' automaticamente a cada push na branch main
- Build da web configurado com `--base-href /viaja-ai-bolso/` para funcionar corretamente no endereço do GitHub Pages
- Publicação automática do resultado do build (pasta build/web) no GitHub Pages via actions/deploy-pages
- Necessário habilitar manualmente 'GitHub Actions' como origem em Settings > Pages do repositório (passo único, feito pela interface do GitHub)
- Link de acesso ao app: <https://viajaaibolso-design.github.io/viaja-ai-bolso/>
- Primeiro deploy confirmado com sucesso (build e publicação concluídos no GitHub Actions) — link já está no ar